

Unitree G1 Research Engineering

Jiankun (Jake) Zhang | Technical summary | July 6-August 3, 2026

Research focus

I am building and evaluating reinforcement-learning systems for Unitree G1 motion imitation, fall recovery, and locomotion. My work combines policy architecture experiments with the less visible engineering required for reproducible distributed training, export, simulation, and evaluation.

What I built

- **Training infrastructure.** Brought Isaac Lab up in rootless Podman on an older host, enabled CUDA and NVIDIA EGL, and established distributed multi-GPU training with reproducible checkpoints, TensorBoard, ONNX/JIT export, and MuJoCo evaluation.
- **Structured-policy studies.** Implemented continuous SonicLite, an FSQ bottleneck ablation, and a more faithful G1 SONIC-style actor. Added bounded Gaussian action noise and distributed PPO gradient accumulation; a two-GPU update combines 196,608 fresh samples.
- **Evaluation tooling.** Built matched-budget analyses, resumed-run event stitching, deterministic videos, and a three-seed observation-noise protocol over five motions and four noise levels.
- **Recovery and locomotion.** Integrated parquet get-up trajectories as an AMP prior, trained an 8,192-environment recovery policy, and profiled a locomotion observation bottleneck down to repeated PhysX material queries.

Selected evidence

- **Continuous latent:** at 5,000 iterations, continuous SonicLite reached final-500 reward 4.827 versus 4.018 for MLP, but used about 31% more actor parameters and only one seed.
- **Quantized variants:** the tested SonicLite-FSQ policies improved some tracking errors but did not beat MLP on overall reward. At the older matched 10k endpoint, reward was 27.96 for MLP versus 21.39 for SonicLite-FSQ.

Evaluation evidence

- **Noise robustness:** across the five-motion protocol, MLP achieved 98.33% success and 37.06 mm MPJPE; SonicLite-FSQ achieved 96.67% and 41.42 mm. This did not support the hypothesized FSQ robustness advantage.
- **Recovery:** the 8,192-environment AMP run completed 10k iterations with 225-step episodes and zero final assistance. A deterministic side-fall video recovered upright in about three seconds; randomized success is not yet measured.
- **Systems result:** vectorizing a critic-side PhysX material observation raised locomotion smoke-test throughput from about 275 to 25,000-29,000 steps/s; the two-GPU full run later sustained about 73,600 steps/s.

Questions I want to pursue

- When do discrete motion representations improve robustness or sample efficiency after controlling capacity, data, seed, and optimizer stability?
- How should large-environment PPO scale mini-batches, KL control, entropy, and action-noise constraints without destabilizing motion quality?
- Can recovery and locomotion policies be evaluated with held-out motions, randomized falls, direct jerk metrics, and sim-to-sim tests that support a defensible robustness claim?

Stack: Python, PyTorch, RSL-RL PPO/AMP, Isaac Lab/Isaac Sim, MuJoCo, ONNX/JIT, Podman, CUDA, distributed training, TensorBoard, profiling.